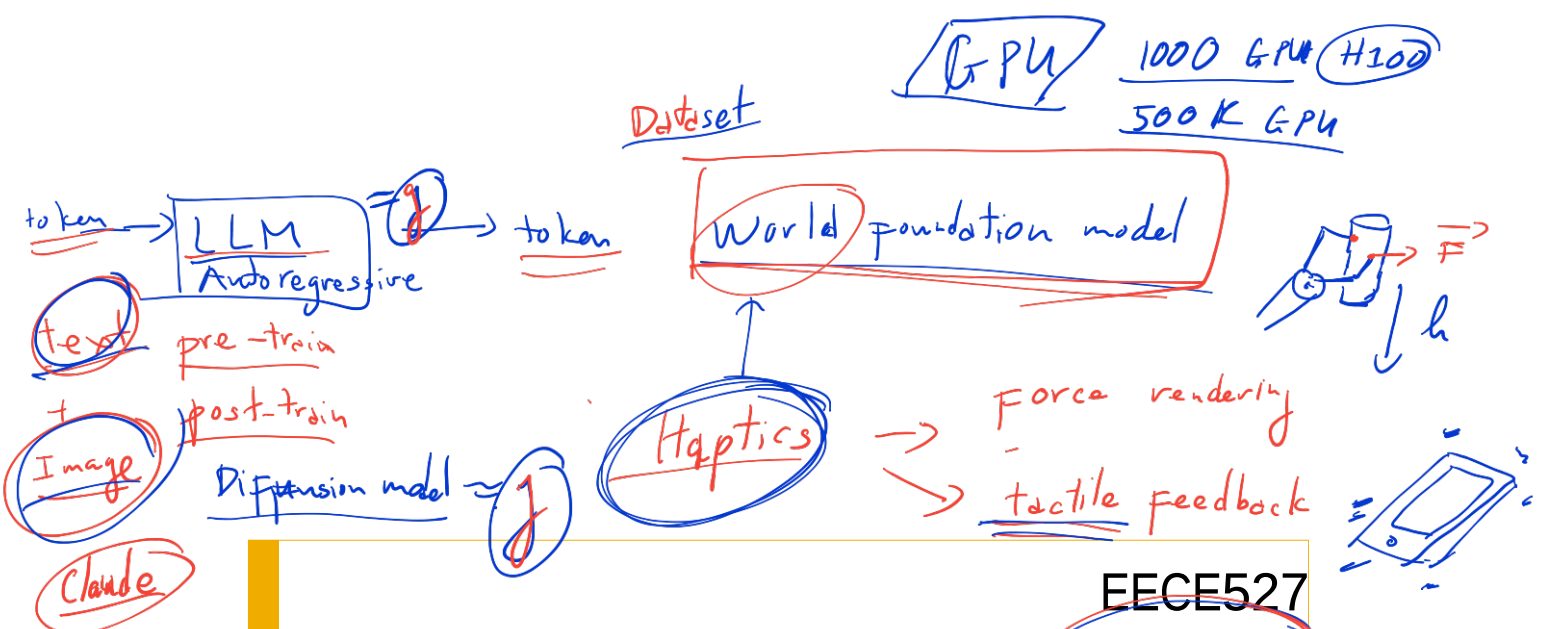




EECE527
INTRODUCTION TO HAPTICS
LECTURE 7: CHAI3D

2011-03-31
In Lee

INTRODUCTION

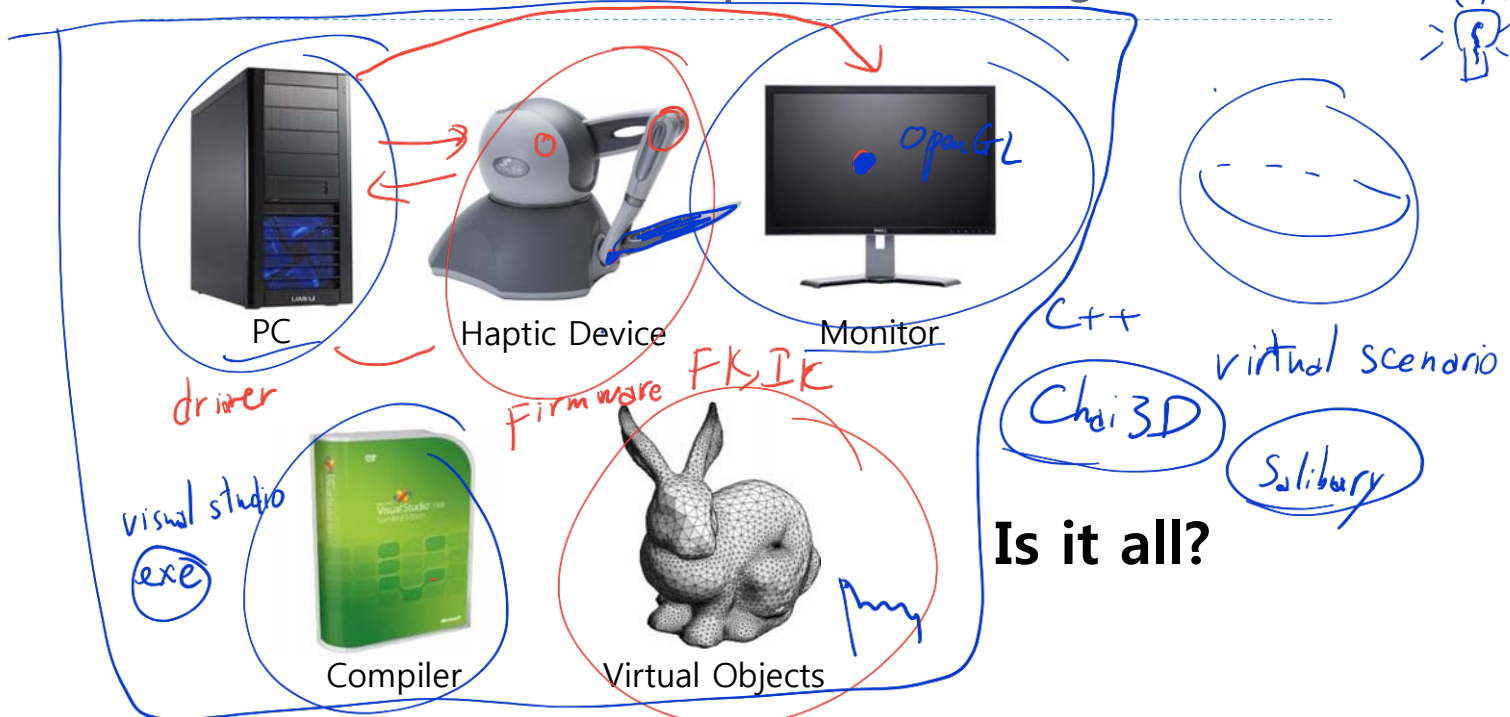
Haptic Rendering

- ▶ Making virtual objects touchable



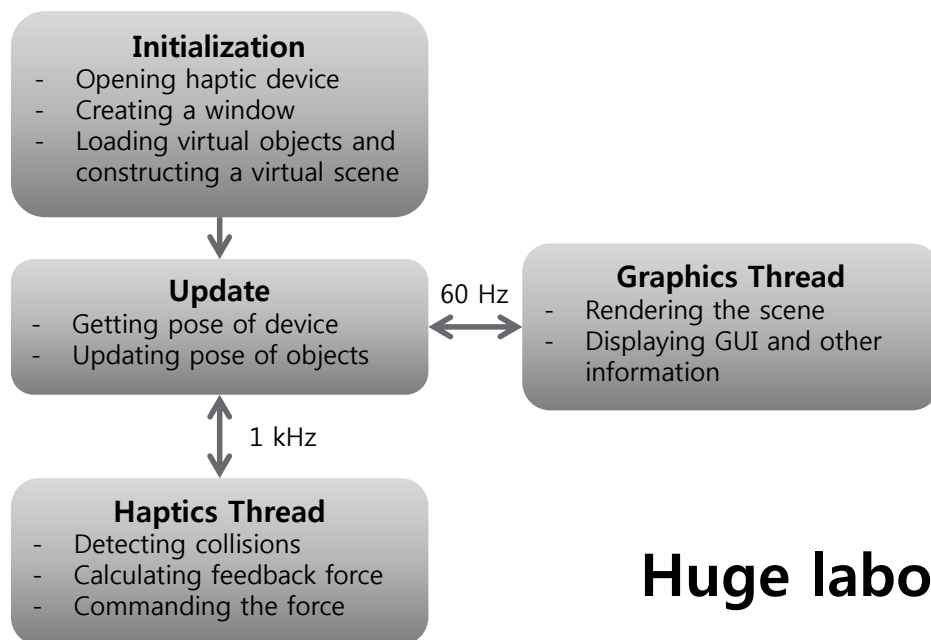
▶ 3

What We Need for Haptic Rendering



▶ 4

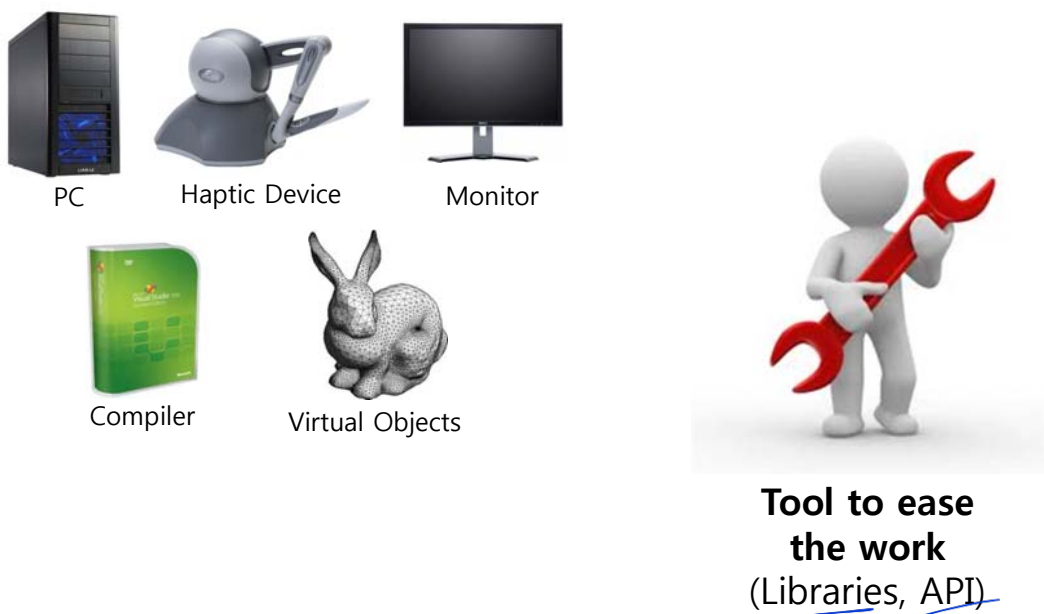
Haptic Rendering Procedures



Huge labor!

► 5

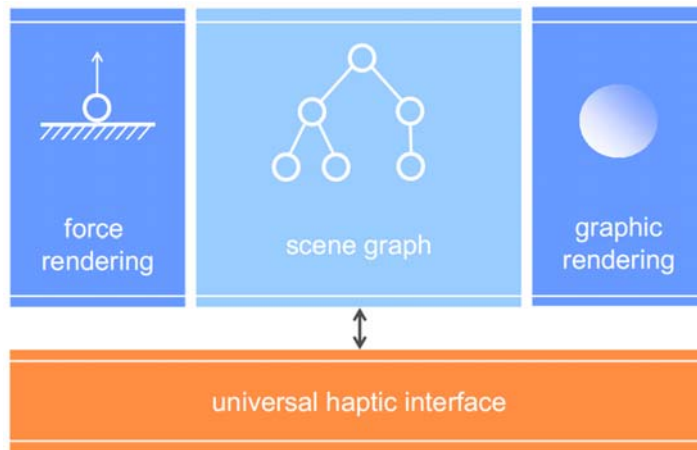
What We Need for Haptic Rendering



► 6

CHAI3D

- ▶ An open source set of C++ libraries for computer haptics, visualization and interactive real-time simulation.
- ▶ The first release is out at 2004 by Artificial Intelligence Lab. of Stanford Univ.



+ ODE: dynamic simulation
GEL: deformable rendering
BASS: sound play

Why Chai3D?

- ▶ Other libraries (e.g., OpenHaptics, Ghost, H3D)
 - ▶ Made by haptic device manufacturers.
 - ▶ Commercial or only have very basic functions
 - ▶ Supports only their own devices.
 - ▶ Cannot be changed.
- ▶ Chai3D
 - ▶ Rich functions
 - ▶ Supports a lot of devices including PHANTOM and Omega.
 - ▶ Platform independent (Windows, Linux, OS X, etc.)
 - ▶ Open source

-

A set of algorithms that detects collisions

A set of algorithms that computes the feedback force
(vector field method and virtual proxy)

High-level interface to haptic devices

Low-level interface to the devices

Special effects such as stick-slip, viscosity, and vibration.

Elements to describe the scene

Matrices, vectors, and constants

Time related things

Object (.obj, .3ds) and
image (.bmp, .tga) loaders

Definitions and includes

2D Image and text

Graphics related components
(vertex, triangle, material, ...)

Display related functions
(viewport, stereo rendering, ..)



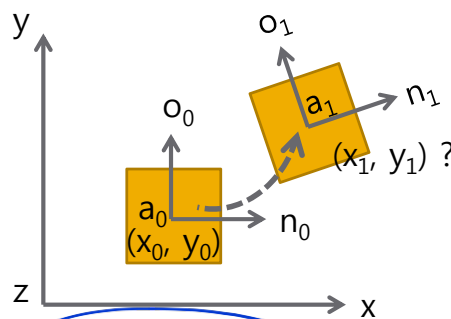
Math

Homogeneous coordinate

4×4 4×1

- ▶ cMatrix3d: 3×3 matrix for rotation
- ▶ cVector3d: 3×1 vector for translation

cMatrix3d * matrix1
 3×3



$$n_1 = \begin{bmatrix} n_x \\ n_y \\ n_z \end{bmatrix} \quad o_1 = \begin{bmatrix} o_x \\ o_y \\ o_z \end{bmatrix} \quad a_1 = \begin{bmatrix} a_x \\ a_y \\ a_z \end{bmatrix}$$

$$R = [n_1 \quad o_1 \quad a_1]$$

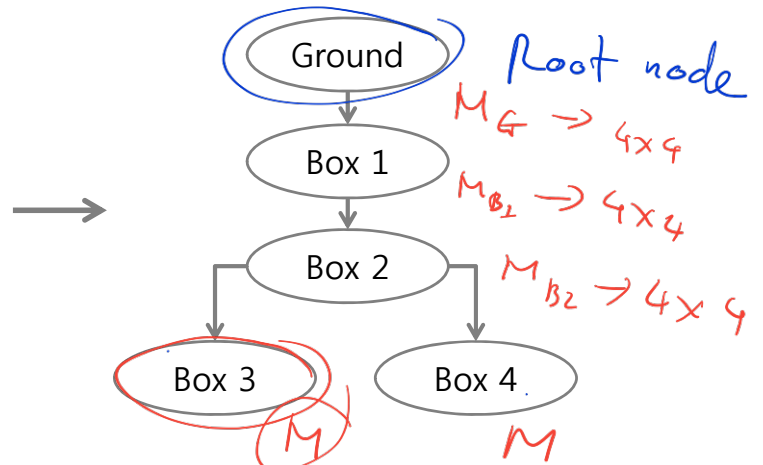
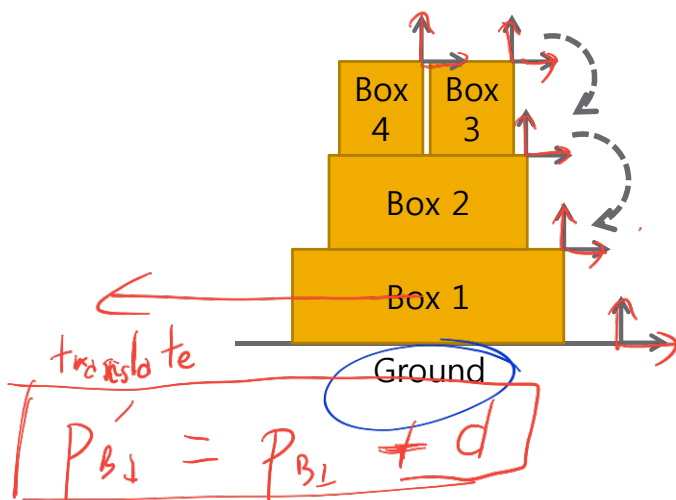
$$t = \begin{bmatrix} \Delta x \\ \Delta y \\ \Delta z \end{bmatrix}$$

- ▶ Most of matrix/vector operations are implemented:
Add, subtract, multiply, divide, transpose, inverse, cross product, etc.
- ▶ cQuaternion: 4×1 vector for rotation
- ▶ cConstants, cMaths: a set of mathematical constants and operations

▶ 11

Scene Graph (1)

- ▶ A way of describing scenes
- ▶ Represents the logical structure of a scene as a tree.
- ▶ Convenient when simulating dynamic objects.



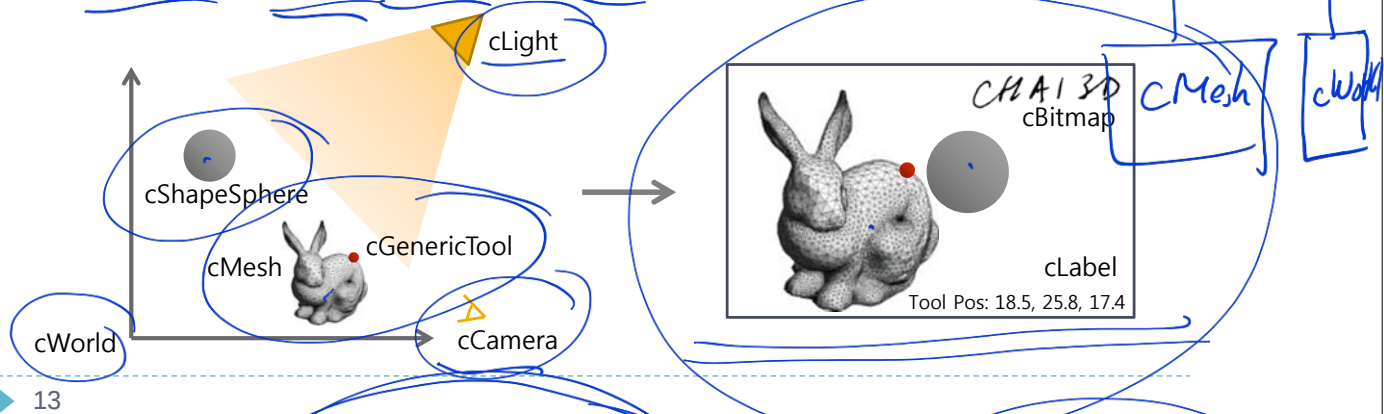
▶ 12

Scene Graph (2)

cGenericObject

abstract class

- ▶ Can have own collision detection algorithm, graphical features (color, transparency, texture) and haptic features (stiffness, friction, special effects)
- ▶ cWorld: root of the scene
- ▶ cMesh: polygonal objects; can be loaded from file
- ▶ cShapeLine, cShapeSphere, cShapeTorus: primitive objects
- ▶ cCamera, cLight: camera and light source for graphics rendering



▶ 13

Claude code

class diagram

OPP

Scene Graph: CMesh

Class

- ▶ Class for virtual objects represented as triangle meshes.
- ▶ A set of vertices, triangles, and texture images.
- ▶ Uses the functions of *file* module.

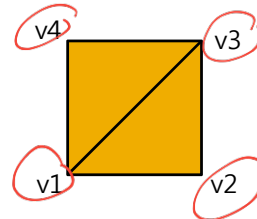
```

Vertices
v 0.0 0.0 0.0
v 2.0 0.0 0.0
v 2.0 2.0 0.0
v 0.0 2.0 0.0

Texture
coord.s
vt 0.0 0.0
vt 1.0 0.0
vt 1.0 1.0
vt 0.0 1.0

Vertex
Normals
vn 0.0 0.0 1.0

Faces
f 1/1/1 2/2/1 3/3/1
f 1/1/1 3/3/1 4/4/1
    
```



CMesh

vertex fragment

Example of Alias wavefront file format (.obj)

▶ 14

Devices & Haptic Tools

- ▶ cHapticDeviceHandler: a manager of haptic devices
- ▶ cGenericHapticDevice
 - ▶ Low-level interface: start/stop, read, and command
 - ▶ cDeltaDevice, cPhantomDevice, etc.: for commercial devices
 - ▶ cVirtualDevice: for virtual device



- ▶ cGenericTool
 - ▶ High-level interface: a set of a device, proxy, collision detection, and force rendering algorithm
 - ▶ cGeneric3dofPointer: a sphere for 3-DOF rendering

x, y, z

▶ 15

Force Algorithm: cPotentialFieldForceAlgo

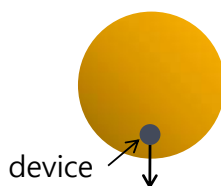
Graphics rendering
Force rendering

→ phong shader

- ▶ Computes the force due to the effects of Haptic Effect module.
- ▶ Actual work is done by *computeInteractions* function of cGenericObject.
- ▶ Effects

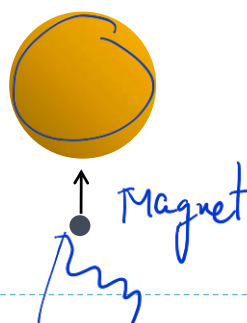
cEffectSurface

Pushes out the device to the nearest point on the surface. Only works with cShapeSphere and cShapeTorus.



cEffectMagnet

Renders magnetic force field. Only works with cShapeSphere and cShapeTorus.



cEffectVibration

Gives sinusoidal vibration. Amplitude and frequency of the vibration are defined in the material of the object.

cEffectViscosity

Renders viscous friction during contact.

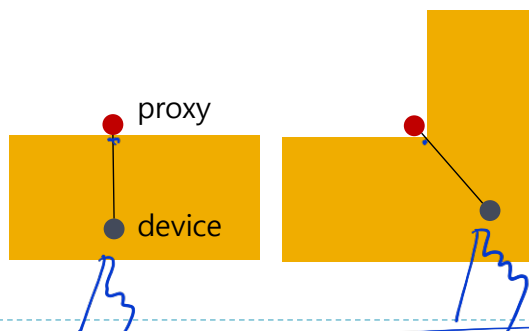
cEffectStickSlip

Renders stick-slip effect of static friction.

▶ 16

Force Algorithm: cProxyPointForceAlgo

- ▶ Compute the force based on the virtual proxy algorithm.
- ▶ Proxy is an avatar of the device located at the ideal position (=nearest point on the surface) of the device.
- ▶ Feedback force is proportional to the distance between the proxy and device.
- ▶ Only works with cMesh objects.
- ▶ Uses computeCollisionDetection function of cGenericObject.



$$L = \frac{1}{2}(x - x_p)^2 + \frac{1}{2}(y - y_p)^2 + \frac{1}{2}(z - z_p)^2 + l_1(A_1x + B_1y + C_1z - D_1) + l_2(A_2x + B_2y + C_2z - D_2) + l_3(A_3x + B_3y + C_3z - D_3) \quad \text{Contact surfaces}$$

$$\nabla L = \begin{bmatrix} 1 & 0 & 0 & A_1 & A_2 & A_3 \\ 0 & 1 & 0 & B_1 & B_2 & B_3 \\ 0 & 0 & 1 & C_1 & C_2 & C_3 \\ A_1 & B_1 & C_1 & 0 & 0 & 0 \\ A_2 & B_2 & C_2 & 0 & 0 & 0 \\ A_3 & B_3 & C_3 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ l_1 \\ l_2 \\ l_3 \end{bmatrix} = \begin{bmatrix} x_p \\ y_p \\ z_p \\ D_1 \\ D_2 \\ D_3 \end{bmatrix}$$

Method of Lagrange multipliers

Collision Detection (1)

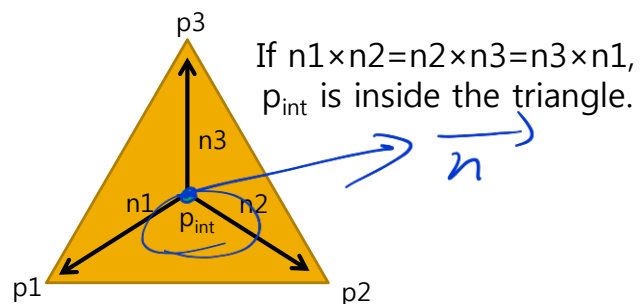
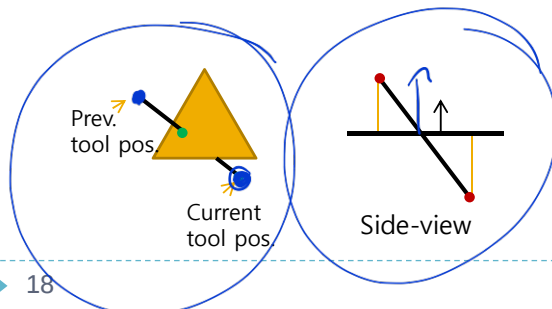
- ▶ Used by cProxyPointForceAlgo

cCollisionBrute

- ▶ Test collisions of the motion segment (a line which connects the previous and current positions of the device) with all triangles in the scene (objects).

- ▶ Uses cIntersectionSegmentTriangle function of cMath.

- ▶ Find the normal vector of the triangle.
- ▶ Find the intersection point.
- ▶ Perform the clockness test.

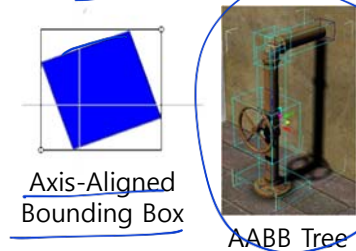


Collision Detection (2)

cCollisionAABB

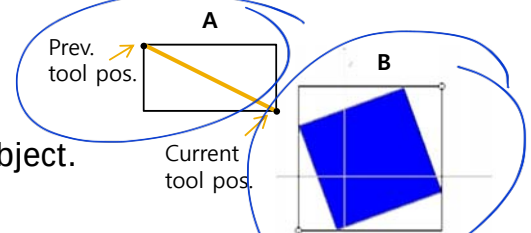
► Detects collisions using Axis-Aligned Bounding Box (AABB) Tree

- AABB is a box that encloses an object and its axis is aligned with the world coordinate.
- All nodes of AABB Tree are AABB that encloses all objects of its children as well as its own one.



► Collision detection procedure

- Make an AABB for the motion segment of the tool in the local coordinate frame of the object.
- From the root, traverse down to the child nodes if there are possible collisions between the AABBs for the segment and object.
- Once reached to the leaf node, perform the line-triangle collision test.

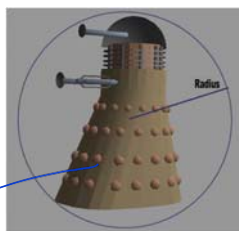


If (Min. Ax > Max. Bx || Min. Ay > Max. By
|| Max. Ax < Min. Bx || Max. Ay < Min. By)
→ A and B are not collide.

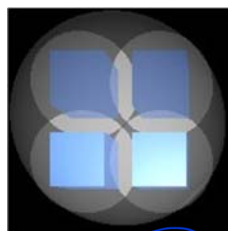
Collision Detection (3)

cCollisionSpheres

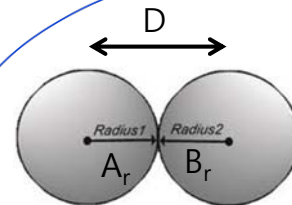
- Detects collisions using Sphere Tree.
- Idea is the same with the AABB Tree, but use bounding spheres instead AABBs.
- No need to be updated when the object is rotated.
- Less compact than AABB Tree.



Bounding Sphere



Sphere Tree



If ($D < A_r + B_r$)
→ A and B are collide.

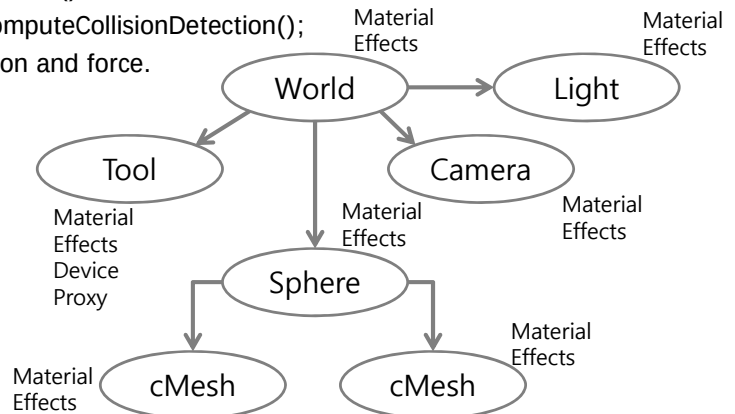
Mesh

Acceleration structure

Basic Flow of Haptic Loop

1. Tool.updatePose();
2. Tool.computeInteractionForces();
 1. Tool.potentialFieldsForceModel.computeForces();
 - For all objects in the scene, computeInteractions();
 - For all effects of the object, computeForce();
 2. Tool.proxyPointForceModel.computeForces();
 - For all objects in the scene, computeCollisionDetection();
 - Calculate the next proxy position and force.

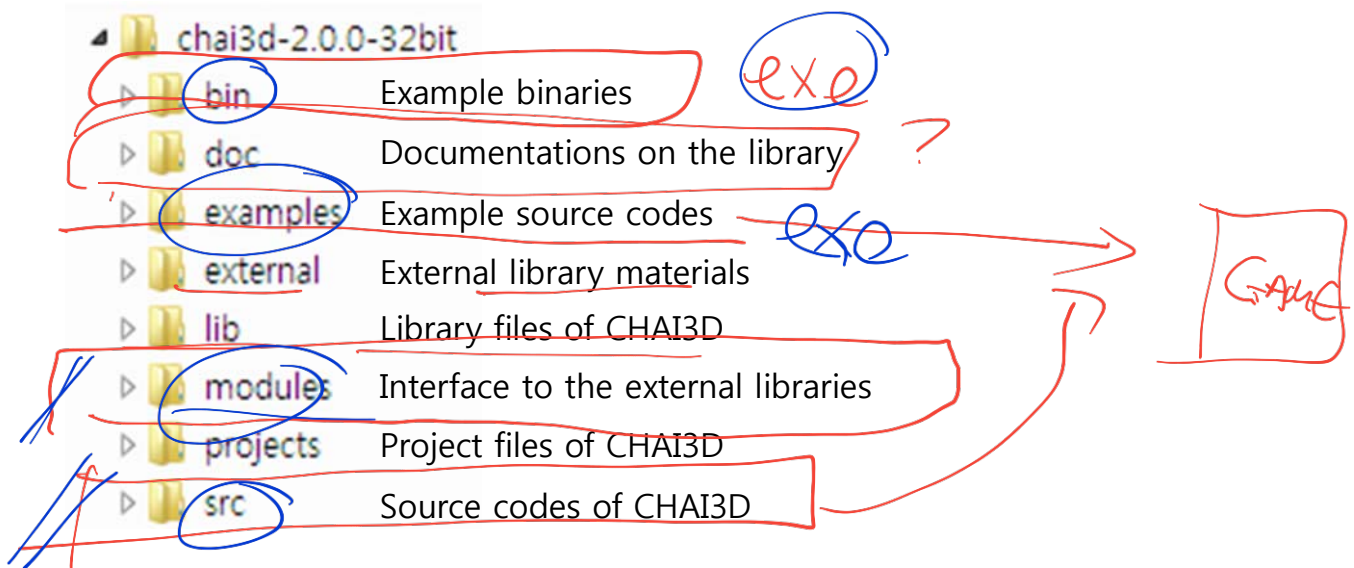
3. Tool.applyForces();



CHAI3D PROGRAMING EXAMPLE

Getting CHAI3D

- ▶ Download CHAI3D from <http://www.chai3d.org/download.html>



▶ 23

Setting Up

▶ Prerequisites

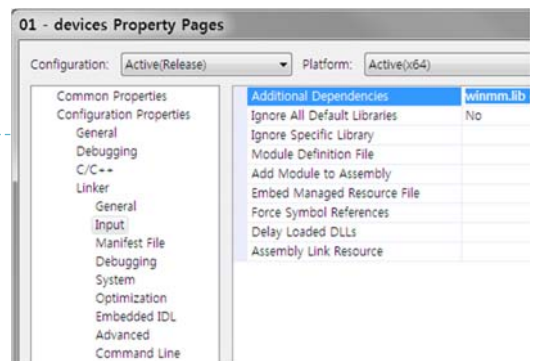
- ▶ PHANToM device driver 4.2
- ▶ OpenHaptics 3.0 Academic Edition
- ▶ CHAI3D 2.0 → *github*

▶ Copy all include files and libraries within your project folder.

- ▶ **.h and .cpp**: \$PROJECT\$/include/
- ▶ **.lib**: \$PROJECT\$/lib/
- ▶ **.dll**: \$PROJECT\$/bin/

▶ Project setting (Project Property pages/Configuration Properties/)

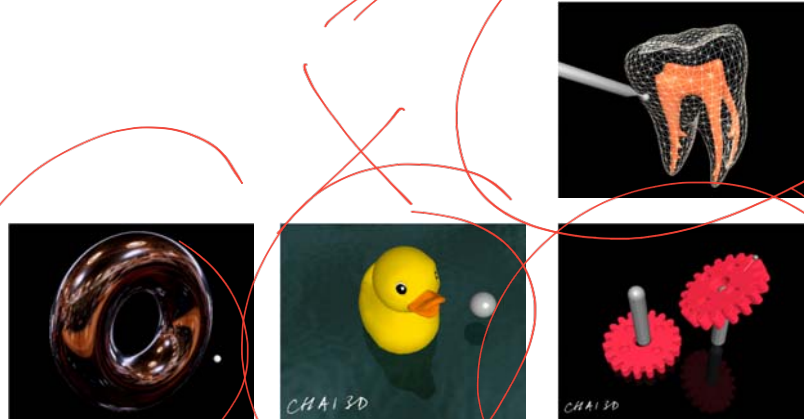
- ▶ [General/Output Directory] Change to "../bin"
- ▶ [C/C++/Additional Include Directories] Add "../include"
- ▶ [Linker/General/Additional Library Directories] Add "../lib"
- ▶ [Linker/Input/Additional Dependencies] Add "winmm.lib glut32.lib chai3d-release.lib"



▶ 24

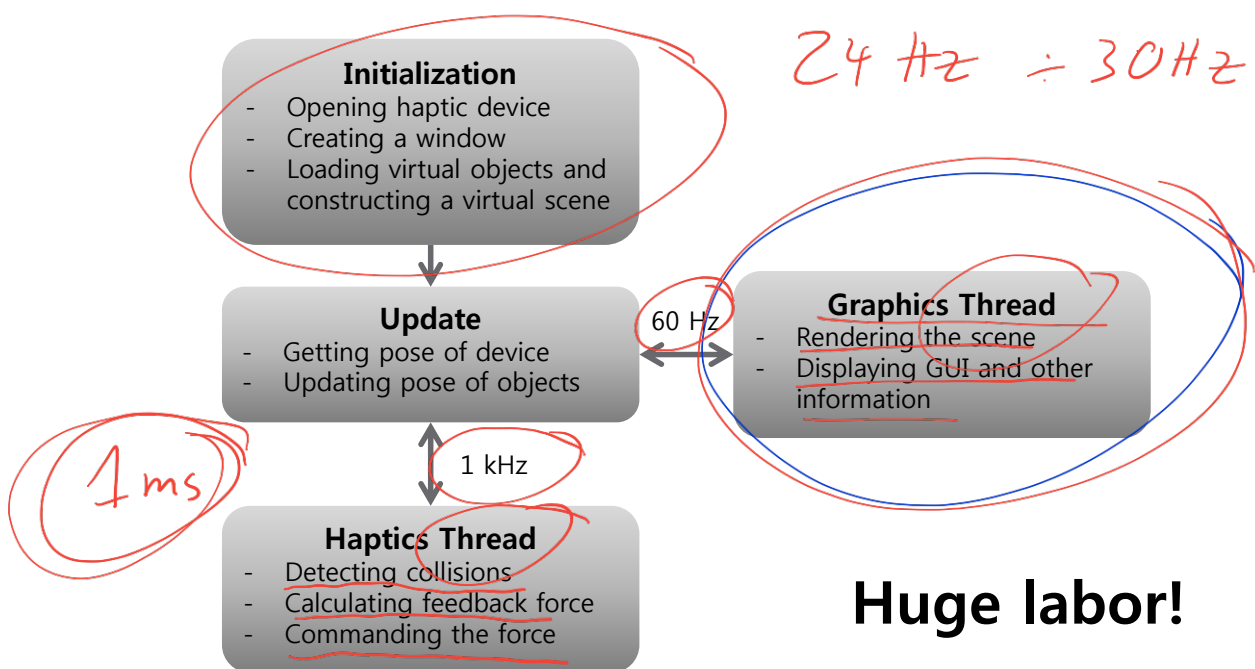
Official Examples

- ▶ CHAI3D provides many examples.
- ▶ Source: [\\$CHAI3D\\$/examples/](#)
- ▶ Binary: [\\$CHAI3D\\$/bin/](#)



▶ 25

Haptic Rendering Procedures



▶ 26

Code Example: Making a Scene

```

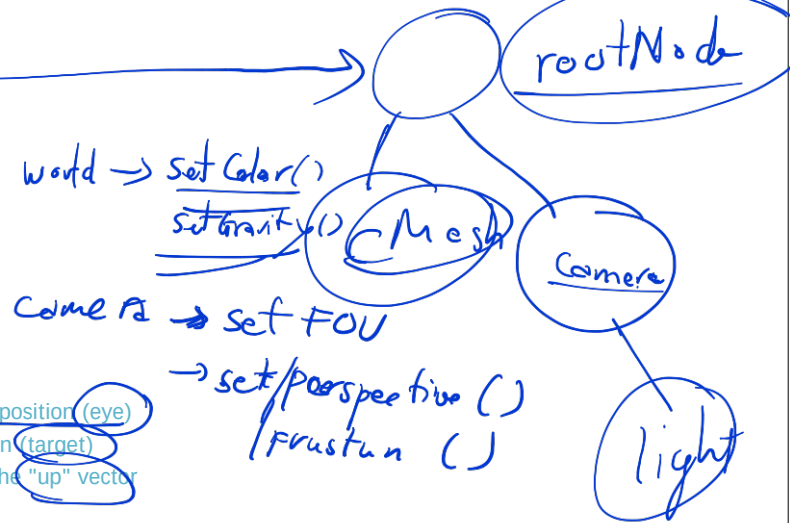
> // create a new world.
> world = new cWorld();

> // set the background color of the environment
> // the color is defined by its (R,G,B) components.
> world->setBackgroundColor(1.0, 1.0, 1.0);

> // create a camera and insert it into the virtual world
> camera = new cCamera(world);
> world->addChild(camera);

> // position and orient the camera
> camera->set( cVector3d( 3.0, 0.0, 0.0), // camera position (eye)
>             cVector3d( 0.0, 0.0, 0.0), // lookat position (target)
>             cVector3d( 0.0, 0.0, 1.0)); // direction of the "up" vector
> camera->setClippingPlanes(0.01, 10.0);

> // create a light source and attach it to the camera
> light = new cLight(world);
> camera->addChild(light); // attach light to camera
> light->setEnabled(true); // enable light source
> light->setPos(cVector3d( 2.0, 0.5, 1.0)); // position the light source
> light->setDir(cVector3d(-2.0, 0.5, 1.0)); // define the direction of the light beam
    
```



27

Code Example: Adding an Image to the Scene

```

> // create a 2D bitmap logo
> logo = new cBitmap();

> // add logo to the front plane
> camera->m_front_2Dscene.addChild(logo);

> // load a "chai3d" bitmap image file
> bool fileload;
> fileload = logo->m_image.loadFromFile("/images/logo.bmp");

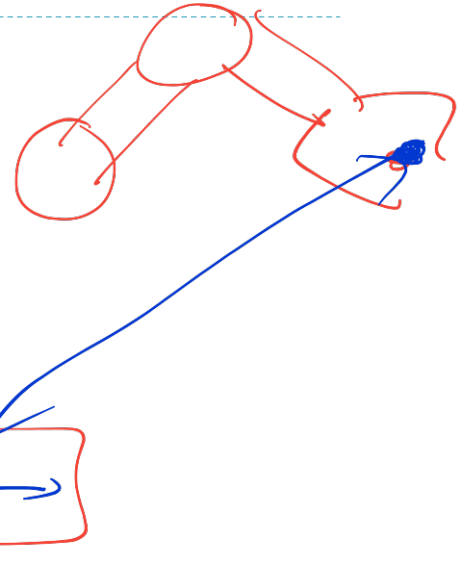
> // position the logo at the bottom left of the screen (pixel coordinates)
> logo->setPos(10, 10, 0);

> // scale the logo along its horizontal and vertical axis
> logo->setZoomHV(0.25, 0.25);
    
```

28

Code Example: Setting a Tool (1)

- ▶ `// create a haptic device handler`
- ▶ `cHapticDeviceHandler handler = new cHapticDeviceHandler();`
- ▶ `cGenericHapticDevice* hapticDevice`
- ▶ `handler->getDevice(hapticDevice, 0);`
- ▶ `cHapticDeviceInfo info;`
- ▶ `if (hapticDevice)`
- ▶ `info = hapticDevice->getSpecifications();`
- ▶ `// create a 3D tool`
- ▶ `tool = new cGeneric3dofPointer(world);`
- ▶ `world->addChild(tool);`
- ▶ `tool->setHapticDevice(hapticDevice);`
- ▶ `tool->start();`
- ▶ `// map the physical workspace of the haptic device to a larger virtual workspace.`
- ▶ `tool->setWorkspaceRadius(1.0);`



▶ 29

Code Example: Setting a Tool (2)

- ▶ `tool->setRadius(0.05);`
- ▶ `// hide the device sphere. only show proxy.`
- ▶ `tool->m_deviceSphere->setShowEnabled(false);`
- ▶ `// set the physical radius of the proxy.`
- ▶ `proxyRadius = 0.05;`
- ▶ `tool->m_proxyPointForceModel->setProxyRadius(0.05);`
- ▶ `tool->m_proxyPointForceModel->m_collisionSettings.m_checkBothSidesOfTriangles = false;`
- ▶ `// read the scale factor between the physical workspace of the haptic`
- ▶ `// device and the virtual workspace defined for the tool`
- ▶ `double workspaceScaleFactor = tool->getWorkspaceScaleFactor();`
- ▶ `double stiffnessMax = info.m_maxForceStiffness / workspaceScaleFactor;`

▶ 30

Code Example: Adding a Mesh to the Scene

```
▶ // create a virtual mesh
  object = new cMesh(world);
  world->addChild(object);

▶ // set the position and orientation of the object at the center of the world
  object ->setPos(0.0, 0.0, 0.0);
  object ->rotate(cVector3d(0.0, 0.0, 1.0), cDegToRad(-10));

▶ // load an object file
  fileload = tooth->loadFromFile(RESOURCE_PATH("/models/object.3ds"));

▶ // compute a boundary box
  object->computeBoundaryBox(true);

▶ // resize object to screen
  object->scale(0.003);

▶ // make an AABB Tree
  object->createAABBCollisionDetector(1.01 * proxyRadius, true, false);

▶ // define a default stiffness for the object
  object->setStiffness(0.8 * stiffnessMax, true);
```

31

obj

AABB

Code Example: Graphics and Haptics Loop

```
▶ Graphics Loop{
  ▶ camera->renderView(displayW, displayH);
  ▶ glutSwapBuffers();
  ▶ }
```

thread

```
▶ Haptics Loop{
  ▶ tool->updatePose();
  ▶ tool->computeInteractionForces();
  ▶ tool->applyForces();
  ▶ }
```

thread

32

Multimedia Timer

- ▶ Good way to run the hapticLoop at every millisecond.

1 ms

```
While(1)
{
    updatePose();
    computeForce();
    applyForces();
}
```

Hold the CPU

```
While(1)
{
    updatePose();
    computeForce();
    applyForces();
    Sleep(0);
}
```

Cannot ensure 1 kHz

```
void CALLBACK hapticsLoop(UINT uID, UINT uMsg, DWORD_PTR dwUser, DWORD_PTR dw1, DWORD_PTR dw2)
{
    updatePose();
    computeForce();
    applyForces();
}
```

ON TOP OF THE MAIN.C

```
#pragma comment(lib, "Winmm.lib")
#include <Windows.h>
```

IN MAIN()

```
UINT m_nTimerID;
DWORD_PTR dwUser;
m_nTimerID = timeSetEvent(1, 0, hapticsLoop, dwUser, TIME_PERIODIC | TIME_CALLBACK_FUNCTION);
```

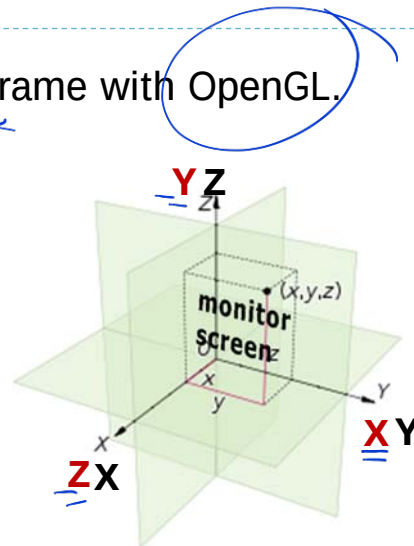
ON EXIT

```
timeKillEvent(m_nTierID);
```

Cautions

- ▶ CHAI3D uses a different coordinate frame with OpenGL.

- ▶ Red: OpenGL axes
- ▶ Black: corresponding CHAI3D axes



- ▶ CHAI3D uses the metric system.
 - ▶ Length: meter (m)
 - ▶ Force: Newton (N)
 - ▶ Stiffness: N/m

References

- ▶ www.chai3d.org
- ▶ CHAI3D documentations (www.chai3d.org/doc/API-documentation.html)
- ▶ Federico Barbagli, “Chai3d: An Open-source Toolkit for Haptic Rendering & Applications,” a presentation material for SIGGRAPH 2005.

Assignment

- ▶ Render an object by yourself!
 - ▶ Get an object from the web.
 - ▶ Load, scale, translate, and rotate the object.
 - ▶ Give stiffness and friction to your object.
 - ▶ You can start from the example code of CHAI3D.
 - ▶ You will get an additional score if you add your own haptic effect to the object.
- ▶ Due date: Apr. 7 (one week later)